

Music and multimedia streaming over the Internet - Lab 4

MEHRAN BAYAT – S320636

May 15, 2026

4 Deliveries

4.1 Ping

4.1.1 To known IP address

1. **Chosen IP:** 130.192.9.250
2. **Exact command issued:** `ping -c 3 130.192.9.250`
3. **List of protocols involved:** Ethernet II, IPv4, and ICMP. The essential protocols for the ping operation were IPv4 and ICMP. Wireshark also showed some ICMPv6 Router Solicitation packets, but those packets were background traffic and were not part of the ping command to 130.192.9.250.

For this part, I captured the traffic on the `enp1s0` interface because the selected IP address was an external address. The command sent exactly three ICMP Echo Request packets, and the destination host replied with three ICMP Echo Reply packets. Therefore, the important capture result was three request-reply ping exchanges.

Choose an Echo Request or an Echo Reply packet and show:

1. **Packet type (Request or Reply):** Echo Request
2. **Source IP address:** 172.16.203.155
3. **Destination IP address:** 130.192.9.250

The selected packet was an ICMP Echo Request sent by my virtual machine to the selected Politecnico IP address. The corresponding Echo Reply packet had the opposite direction, from 130.192.9.250 to 172.16.203.155. This confirmed that the selected host was reachable from the Crownlabs VM during the experiment.

4.1.2 To named host

1. **Chosen host name:** This part was not captured in the performed work.
2. **Exact command issued:** This part was not executed in the performed work. The command would have the form `ping -c 3 <host-name>`, for example `ping -c 3 leonardo.polito.it`.
3. **IP address of the chosen host:** Not recorded, because the named-host capture was not performed.
4. **Describe the additional packet types observed with respect to exercise ??:** Compared with pinging a known IP address directly, pinging a host name would also require DNS packets before the ICMP packets. This is because the system must first translate the host name into an

IP address. After the DNS response is received, the normal ICMP Echo Request and Echo Reply packets would appear, as in the known-IP ping capture.

4.2 Netcat

4.2.1 TCP

1. Report the commands used to run both the Netcat client and the Netcat server

- **Server:** `nc -l 127.0.0.1 8080`
- **Client:** `nc 127.0.0.1 8080`

I used the loopback interface `lo` in Wireshark because both the client and the server were running on the same machine and communicating through `127.0.0.1`. After the capture, I used the display filter `tcp.port == 8080` to isolate the Netcat traffic from the background Crownlabs/VNC packets.

2. Identify the packets used to initiate and close the TCP connection. Provide a screenshot or a list of them:

The TCP connection was initiated with the standard three-way handshake:

```
55760 -> 8080 [SYN]
8080 -> 55760 [SYN, ACK]
55760 -> 8080 [ACK]
```

In this capture, port 8080 was the fixed server port, while port 55760 was the temporary client source port automatically assigned by the operating system.

During the message exchange, Wireshark also showed a data packet carrying the Netcat text:

```
8080 -> 55760 [PSH, ACK]
55760 -> 8080 [ACK]
```

The connection closing packets observed in my capture were:

```
8080 -> 55760 [FIN, ACK]
55760 -> 8080 [ACK]
55760 -> 8080 [FIN, ACK]
8080 -> 55760 [RST]
```

The final packet was a reset instead of a clean final ACK because the Netcat session was stopped manually using the terminal. This can happen when one side is interrupted instead of completing a fully graceful TCP close. The important point is that TCP used explicit control packets to open and close the connection.

3. Write here the source port of the packets going in the direction client → server:

Source port client → server: 55760

4.2.2 UDP

1. Report the commands used to run both the Netcat client and the Netcat server

- **Server:** `nc -u -l 127.0.0.1 8080`
- **Client:** `nc -u 127.0.0.1 8080`

These are the corresponding UDP commands. The option `-u` changes Netcat from TCP mode to UDP mode.

2. What is the main difference in the number of packets when using UDP instead of TCP? Which packets are new, and which ones are missing?

UDP normally produces fewer packets than TCP for the same simple message exchange, because UDP does not create a connection and does not close a connection. In TCP, Wireshark shows the three-way handshake, data packets, acknowledgments, and connection termination packets. In UDP, the handshake packets and the closing packets are missing.

The packets missing in UDP are TCP-specific packets such as:

SYN
SYN, ACK
ACK handshake
FIN, ACK
RST

The packets that appear instead are UDP datagrams. These datagrams carry the message data directly without a connection setup phase.

3. What are the differences between a TCP and a UDP packet? Could you briefly explain why?

TCP and UDP are both transport-layer protocols, but they behave differently. TCP is connection-oriented, so it uses sequence numbers, acknowledgment numbers, flags, and a window field. This allows TCP to provide reliable and ordered byte-stream communication.

UDP is connectionless, so its header is simpler. A UDP packet mainly contains source port, destination port, length, checksum, and data. UDP does not guarantee delivery, ordering, or retransmission. This is why UDP packets do not have TCP flags such as SYN, ACK, FIN, or RST.

Aspect	TCP	UDP
Connection	Connection-oriented	Connectionless
Setup	Uses SYN, SYN-ACK, ACK	No handshake
Reliability	Acknowledgments and sequence numbers	No built-in reliability
Closing	FIN/ACK or RST packets	No formal closing
Header	More complex	Simpler

4. Do you notice any strange behavior in the order of the packets?

A possible strange behavior in UDP is related to the fact that there is no real connection. If the server/listener tries to send the first message, it may not immediately know the client's address until it receives a datagram from the client. Because of this, the order of visible UDP datagrams may look less structured than TCP. In TCP, after the client connects, both sides have an established connection. In UDP, each datagram is independent.

In the work performed here, the exact UDP packet table was not captured, so I do not report exact UDP packet numbers. The explanation above is the accurate protocol-level comparison required for the UDP part.

4.3 Web

4.3.1 HTTP Server

1. Briefly report the type of packets observed in Wireshark

For the HTTP server, I ran:

```
python3 test_server.py
```

Then I opened the page in Firefox using:

```
http://localhost:8080
```

In Wireshark, I captured the traffic on the loopback interface `lo`. The observed packets were TCP and HTTP packets between `127.0.0.1` and `127.0.0.1`. The important HTTP packets were:

```
GET / HTTP/1.1
HTTP/1.0 200 OK
```

```
GET /favicon.ico HTTP/1.1
HTTP/1.0 404 File not found
POST /login HTTP/1.1
```

The POST /login packet was the most important packet because it contained the login data typed in the browser.

2. Could you find the username and password typed in the browser among the packets?

Yes. In the HTTP capture, I could find both the username and the password inside the packets.

The values used in the experiment were:

```
Username: mehran
Password: test123
```

3. If so, where? If not, why?

I found the username and password inside the HTTP POST /login request payload. Wireshark showed the request as plain HTTP, and the body was visible as line-based text data:

```
{"user": "mehran", "pwd": "test123"}
```

This happened because HTTP does not encrypt the application data. Therefore, the login information was visible in clear text in Wireshark. The server terminal also confirmed the same result by printing:

```
Login tentative for user: mehran, with password: test123
```

4.3.2 HTTPs Server

1. Briefly report the type of packets observed in Wireshark

For the HTTPS server, I ran:

```
python3 test_server_https.py
```

Then I opened the page in Firefox using:

```
https://localhost:8080
```

Firefox showed a security warning because the local server used a self-signed certificate. I accepted the risk and continued, as expected for this lab. Then I entered the same credentials as before.

In Wireshark, I filtered the traffic using:

```
tcp.port == 8080
```

I observed TCP packets on localhost port 8080. Unlike the HTTP capture, applying the display filter `http` showed zero readable HTTP packets. This means the HTTP content was not visible as plain HTTP in Wireshark.

2. Could you find the username and password typed in the browser among the packets?

No. In the HTTPS capture, I could not find the username and password in the Wireshark packet payload.

The server terminal showed that the login reached the server:

```
Login tentative for user: mehran, with password: test123
```

However, Wireshark did not show the credentials in clear text.

3. If so, where? If not, why?

I could not find the username and password in Wireshark because HTTPS encrypts the HTTP payload using TLS. The browser and server still exchanged the login data, but the data was encrypted before being transmitted over the network. The server could read the values only after decrypting the HTTPS request.

This is the main difference compared with the HTTP part. With HTTP, the packet body was visible as:

```
{"user":"mehran","pwd":"test123"}
```

With HTTPS, the `http` display filter showed no readable HTTP packets, and the username/password were not visible in the packet bytes. Therefore, HTTPS protected the credentials from being read directly in Wireshark.